Voice Interview Assistant and Colab Benchmark

Run a voice-based mock interview assistant in Google Colab, then optionally benchmark its speech-to-text (STT) and large language model (LLM) processing on CPU and NVIDIA T4 runtimes.

The application uses Whisper tiny for transcription, Llama 3.2 3B through Ollama for feedback, and external gTTS for spoken replies. Processing runs in the Colab runtime; no local GPU is required.

The `ScreenShot` directory contains screenshots illustrating application setup, benchmark execution, and example outputs.

1. Run the mock interview application

1. Open [Google Colab](https://colab.research.google.com/) and create a new notebook.
2. Select **Runtime → Change runtime type**. Choose **T4 GPU** under the hardware accelerator options, then click **Save**. [Screenshot: `ScreenShot/RunTime Selection.png`]
3. Open `mock_interview.py` and copy its complete contents into the first Colab code cell. Click "Code" from the Menu in case code cell is not visible.
4. Click the cell's **Run** button (▶). Wait for dependency installation, model downloads, and server initialization to finish. [Screenshot: `ScreenShot/Mock Interview Script Run.png`]
5. The output will display **BEHAVIORAL INTERVIEW COACH READY!** followed by a URL. Click that URL to open the interview interface. [Screenshot: `ScreenShot/Mock Interview Script Output.png`]
6. Click **Begin Behavioral Interview**, then **Start Answer**. Allow microphone access when prompted, speak your answer, and use the recording control to finish it. The application will transcribe your answer and generate written and spoken feedback. [Screenshot: `ScreenShot/Mock_Interview_Session_Started.png and ScreenShot/Mock_Interview_Feedback`]
7. Keep the Colab runtime connected while using the application. If the runtime is restarted, rerun the application cell and open the newly printed URL.

T4 availability and session limits depend on Colab's current resource allocation. See the [Colab FAQ](https://research.google.com/colaboratory/faq.html). Internet access is required for installation, model downloads, and gTTS.

2. Run the benchmarking script

Use the `benchmarking_script.py` supplied alongside this README. Each CPU/T4 result archive also contains the script captured during that run. The three `COLAB_...` settings below control runtime selection, CPU archive reuse, and automatic downloading.

Use two code cells in the same notebook:

- **Cell 1:** The complete mock interview application script. Run this first to install dependencies, download the models, start Ollama, and initialize Whisper. Rerun the application setup described in Section 1 using the selected benchmark runtime.
- **Cell 2:** The complete benchmarking script, pasted into a new code cell below Cell 1.

Run the **CPU benchmark first**, download its ZIP, then run the **T4 benchmark using that CPU ZIP**. This allows the T4 test to reuse the exact saved input recordings from the CPU test. No microphone recordings or interview-interface interaction are needed for benchmarking.

A. CPU benchmark

1. Download any results you want to keep before changing the runtime.
2. Select **Runtime → Change runtime type**, set the hardware accelerator to **CPU / None**, and save.
3. Run **Cell 1** again in the CPU runtime. Wait until application setup finishes. You do not need to open the interview URL.
4. Paste the entire benchmarking script into **Cell 2**. Click "Code" from the Menu to open Cell 2.
5. Set the following values near the top of that script:

   ```python
   COLAB_BACKEND = "cpu"
   COLAB_CPU_RESULTS_ZIP = None
   COLAB_DOWNLOAD_RESULTS = True
   ```

6. Run **Cell 2**. Do not use the interview interface or send other Ollama requests while the benchmark runs.
7. Wait for completion. A successful suite records one first-request observation, four warm-up requests (two per workload), and 40 measured requests (20 short and 20 longer). [Screenshot: `ScreenShot/CPU_Run_Output.png`]
8. Download the generated CPU ZIP before switching runtimes. Its name follows this pattern:

   ```text
   pm_benchmark_v3_cpu_<timestamp>_<run_id>.zip
   ```

With `COLAB_DOWNLOAD_RESULTS = True`, the script requests a browser download automatically. If no download appears, follow the manual download instructions below.

B. T4 GPU benchmark

1. Confirm that the CPU results ZIP has been downloaded to your computer.
2. Select **Runtime → Change runtime type**, choose **T4 GPU**, and save.
3. Run **Cell 1** again to initialize the application and models in the new T4 runtime. Wait until setup finishes.
4. In Colab's left sidebar, open **Files** and use **Upload** to upload the CPU results ZIP. Keep it zipped. [Screenshot: `ScreenShot/How to Upload the zip file.png`]
5. In **Cell 2**, update the settings: [Screenshot: `ScreenShot/Add upload zip file path.png`]

   ```python
   COLAB_BACKEND = "t4"
   COLAB_CPU_RESULTS_ZIP = "/content/pm_benchmark_v3_cpu_<timestamp>_<run_id>.zip"
   COLAB_DOWNLOAD_RESULTS = True
   ```

   Replace the example path with the actual path of the uploaded CPU ZIP. You can right-click the uploaded file in the Files pane and select **Copy path**.

6. Run **Cell 2**. The script checks for a T4 GPU and reuses the CPU archive's saved audio. Changing `COLAB_BACKEND` alone does not change Colab's hardware; select the runtime in the menu first. [Screenshot: `ScreenShot/T4_GPU_Output.png`]
7. Wait for completion, then download the generated T4 ZIP:

   ```text
   pm_benchmark_v3_t4_<timestamp>_<run_id>.zip
   ```

Keep both CPU and T4 ZIP files for analysis and reproducibility.

C. Download results manually

1. Find the **RESULT ARCHIVE:** line in the cell output. It gives the exact ZIP path.
2. Open Colab's **Files** sidebar and navigate to `/content/pm_benchmark_results/`.
3. Refresh the file list if necessary.
4. Right-click the required `.zip` file and select **Download**.

Results are stored in the runtime, so download them before disconnecting, deleting, or switching runtimes.

3. What the benchmark measures

The benchmark uses five short and five longer synthetic answers, each repeated four times. CPU and T4 use identical saved audio files.

It records:

- Whisper transcription latency and real-time factor, based on actual recording duration.
- Prompt-assembly time.
- LLM request latency through completion of the response, including the final Ollama message.
- Client-observed time to first nonempty response content.
- Ollama-reported generation throughput and token counts.
- Combined STT + LLM latency and runtime diagnostics.
- Reply text-to-speech latency (`tts_ms`).
- Text feedback preparation latency (`pipeline_ms`).
- Audio feedback preparation latency (`feedback_audio_ready_ms`).

Report results separately for short and longer answers, using the 20 measured requests per workload after warm-up. The first-request observation and four warm-up requests are excluded from these statistics. Fields ending in `_ms` are measured in milliseconds; divide by 1,000 to convert them to seconds.

4. Troubleshooting

- **Ollama is unavailable:** Confirm that Cell 1 finished successfully in the current runtime before running Cell 2.
- **CPU/T4 mismatch:** Select the correct hardware in Colab and rerun Cell 1. The benchmark deliberately rejects a CPU condition on a GPU runtime or a T4 condition without a T4.
- **CPU ZIP not found:** Upload it again after switching runtimes and set `COLAB_CPU_RESULTS_ZIP` to its exact uploaded path.
- **No automatic ZIP download:** Use the manual download steps above.

A ZIP may be saved even after a failure. Check that `run_status.json` reports `"state": "completed"` before treating it as a complete benchmark run.